FIME
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UANL
UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA
UNIDAD 3 · DISEÑO ARQUITECTÓNICO DEL SISTEMA

Informe de Arquitectura

Diseño arquitectónico del sistema: Caso Eventos Nova

Materia: Componentes de lógica de negocios
Ing. Miguel Ángel Gómez Marroquín

Equipo 16

Oscar Felipe Renaut Vega (2226683)
Diana Lucía Hernández Fraire (2226678)
José Tadeo Cruz Bazaldúa (2226710)

Octubre de 2026

Índice

Índice 2

1. Alcance de la arquitectura 5

1.1 Delimitación del proceso de negocio 5

1.2 Actores del sistema 5

1.3 Casos de uso operativos seleccionados (Etapa 1) 6

2. Comparación y selección del patrón de lógica de negocio 7

2.1 Matriz comparativa obligatoria (Etapa 2) 7

2.2 Justificación formal de la elección 9

3. Mapa de dominio 10

3.1 Lenguaje ubicuo (glosario técnico de dominio) 10

3.2 Clasificación de entidades y Value Objects 10

3.3 Fichas técnicas de Aggregates 11

Aggregate Root operativo: Evento 11

Aggregate Root financiero: Cuenta de Pagos 11

3.4 Decisiones descartadas del mapa de dominio 13

4. Service Layer (Etapa 4) 14

4.1 Propósito de esta etapa 14

4.2 Casos de uso que orquesta la capa de aplicación 15

4.3 Decisiones de diseño autorizadas 15

4.4 Recorrido común de un servicio de escritura 17

4.5 Tabla oficial de Application Services (CU-01 a CU-05) 17

4.6 Detalle de cada operación 20

4.7 Servicios complementarios: Scheduler y manejadores de eventos 22

4.8 Interfaces que usa la capa de aplicación 25

4.9 Estrategia de errores y resultados 26

4.10 Por qué los servicios no absorben las reglas 26

4.11 Revisión obligatoria de la Etapa 4 27

5. Repository y Unit of Work (Etapa 5) 29
5.1 Definición de interfaces del patrón Repository 29
5.2 Alcance transaccional y Unit of Work (IUnitOfWork) 30
5.3 Contraste con los requisitos de la capa de aplicación 31

6. CQRS básico (Etapa 6) 33
6.1 Cómo se separan escritura y lectura 33
6.2 Tabla oficial de Commands y Queries 33
6.3 Dependencias y relación con el resto del diseño 34
6.4 Contratos de DTOs y ViewModels 36
Envoltorio de resultado 36
Commands y su salida 36
Queries y ViewModels de lectura 38
6.5 Recorrido de una Query y consecuencia de la consistencia eventual 39
6.6 Por qué no se duplica la base de datos física 39
6.7 Qué partes no necesitan esta separación 39
6.8 Control de coherencia 39

7. Revisión SOLID (Etapa 7) 41
7.1 Evaluación formal de los cinco principios 41
7.2 Observaciones complementarias desde la capa de aplicación 42

8. Diagramas de secuencia (Entregable 03) 43
8.1 Qué muestran y cómo leerlos 43
8.2 Diagrama 1: CU-03 Formalizar contrato de evento (flujo exitoso) 44
Recorrido paso a paso 46
Límites transaccionales 46
Consistencia eventual 47
Por qué userId y rol viajan como metadatos de auditoría 47
8.3 Diagrama 2: CU-04 Procesar cambio de alcance (cambio extemporáneo no viable) 47
Recorrido paso a paso 49
Límites transaccionales y consistencia eventual 49
Por qué “No garantizado” se modela como Result de dominio 49

8.4 Coherencia con las tablas de Service Layer y CQRS 50

9. Registros de decisiones de arquitectura (ADR) 51

9.1 ADR-001: Patrón de lógica de negocio, Domain Model 51

9.2 ADR-002: Mecanismo de control de acceso (RBAC) 52

9.3 ADR-003: Estrategia de auditoría e historial inmutable 52

9.4 ADR-004: despacho de eventos en memoria (propuesta del Rol 2) 53

10. Hallazgos y puntos pendientes 55

10.1 Propuestas que cambian el trabajo de Rol 1 55

10.2 Datos que las fuentes no traen 55

10.3 Riesgos de diseño aceptados 56

10.4 Actualización tras integrar los capítulos de Rol 1 y Rol 3 56

Hallazgos anteriores: estado actual 56

10.5 Mitigaciones y Decisiones de Coordinación entre Capas 57

11. Conclusiones de la capa de aplicación 59

Anexo A: Diagrama de clases UML de la arquitectura (Entregable 02) 60

A.1 Código fuente PlantUML 61

Anexo B: Código Base Boilerplate en C# (Entregable 06) 64

Anexo C: Bitácora Oficial de IA (Entregable 05) 66

1. Alcance de la arquitectura

1.1 Delimitación del proceso de negocio

El diseño arquitectónico formaliza el “Ciclo de Gestión Operativa y Financiera de Eventos Nova”, abarcando desde la captura de la solicitud de cotización, la asignación preliminar y confirmación de proveedores bajo acuerdos de nivel de servicio (SLA), la formalización del contrato comercial, el procesamiento de modificaciones extemporáneas de alcance, hasta la liquidación del saldo o autorización de excepciones de pago previa a la ejecución.

1.2 Actores del sistema

Actor	Responsabilidad
Cliente	Solicita cotizaciones, autoriza incrementos de costo, confirma número final de asistentes y notifica cancelaciones.
Coordinador	Administra el catálogo operativo, valida disponibilidad de proveedores, gestiona sustituciones y formaliza contratos.
Proveedor	Confirma disponibilidad, tarifas y condiciones comerciales dentro de la ventana estricta de 48 horas.
Administración	Supervisa anticipos, valida la liquidación de saldos y posee la facultad exclusiva de autorizar excepciones financieras de pago.

1.3 Casos de uso operativos seleccionados (Etapa 1)

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Solicitar cotización de evento	Cliente / Coordinador	BR-01 (15 días anticipación) BR-05 (Rango 20-300 invitados) BR-15 (Máximo 15 proveedores)	Cotización inicial generada; o alerta por anticipación insuficiente; o evaluación especial de capacidad.
CU-02	Confirmar respuesta de proveedor	Proveedor / Coordinador	BR-04 (SLA estricto 48h) BR-10 (Sustituto equivalente) BR-17 (Exclusividad Coordinación)	Proveedor confirmado; o marcado como “Sin confirmar” por vencimiento de SLA; o reasignado por coordinador.
CU-03	Formalizar contrato de evento	Coordinador	BR-07 (4 condiciones obligatorias: propuesta aceptada, datos completos, servicios confirmados y anticipo cubierto)	Contrato formalizado y monto contractual fijado; o bloqueo preventivo por condición faltante.
CU-04	Procesar cambio de alcance	Cliente / Coordinador	BR-02 (Límite ordinario 7 días) BR-06 (Hasta 2 cambios gratuitos) BR-08 (Actualización de cotización) BR-12 (Cambio fuera de plazo)	Cambio aplicado sin costo; o cotización de incremento pendiente de aceptación; o rechazo como “No garantizado”.
CU-05	Liquidar saldo o autorizar excepción	Cliente / Administración	BR-09 (Liquidación 3 días antes) BR-16 (Exclusividad Administración)	Evento en estado “Pagado”; cancelación contractual por mora; o excepción de pago auditada.

2. Comparación y selección del patrón de lógica de negocio

2.1 Matriz comparativa obligatoria (Etapa 2)

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Las validaciones del SLA de 48h (BR-04) y la gratuidad condicional de cambios (BR-06) requerirían múltiples condicionales duplicados en endpoints.	Empaquetaría toda la lógica en clases de tabla gigantescas altamente acopladas al esquema relacional de la base de datos.	Encapsula las reglas complejas; la entidad Evento sabe por sí misma si un cambio es extemporáneo (BR-02) sin que el servicio externo lo calcule.	BR-04, BR-06, BR-02, BR-12
Cambios esperados	Modificar la lógica para calcular devoluciones de anticipo afectaría directamente a los controladores o capas de UI.	Fácil de actualizar consultas, pero los cálculos mixtos (ej. gastos comprometidos en BR-14) ensuciarían el módulo.	Si cambia la penalización o la retención de anticipo (BR-14), la modificación ocurre de manera aislada en el Value Object financiero.	BR-14, BN-14
Interacción entre conceptos	Gestionar la reasignación de un sustituto y actualizar la cotización requiere scripts largos llamando a múltiples tablas separadas.	Los módulos de tabla para Proveedor y Evento estarían altamente acoplados para coordinar actualizaciones de costo total (BR-08).	Se crean Aggregates claros: la raíz Evento coordina a sus asignaciones y orquesta el recálculo financiero de forma desacoplada.	BR-10, BR-08, Glosario
Estados e invariantes	Fácil de saltarse invariantes (ej. permitir formalizar sin anticipo), ya que el control del estado depende del script en turno.	Los estados se controlan solo antes de guardar en la BD, lo que permite que el objeto en memoria esté en un estado inválido.	La formalización garantiza sus invariantes: el contrato no se genera si las 4 condiciones obligatorias (BR-07) no se cumplen en memoria.	BR-07, BN-06, BPMN

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Control de roles (RBAC)	Los chequeos de autorización (if rol == Administracion) estarían mezclados con la lógica operativa en los endpoints.	La validación de roles tendría que incrustarse en los métodos del módulo de base de datos, violando responsabilidades.	La lógica de validación de rol (BR-16, BR-17) se inyecta desde la Service Layer, mientras que el Dominio permanece agnóstico.	BR-16, BR-17, FR-16

2.2 Justificación formal de la elección

Para Eventos Nova se adopta Domain Model. El análisis del BRD demuestra que la operación está gobernada por transiciones de estado complejas e interdependientes (SLA de 48h, límite de 7 días naturales para cambios y 4 condiciones duras de formalización). Transaction Script dispersaría las retenciones de anticipo en código duplicado y Table Module limitaría el diseño a la estructura de datos física. Domain Model encapsula el comportamiento en entidades ricas, manteniendo la Service Layer como un orquestador libre de lógica de negocio (ADR-001).

3. Mapa de dominio

3.1 Lenguaje ubicuo (glosario técnico de dominio)

- **Asignación de Proveedor:** Entidad interna del agregado Evento que controla el estado operativo (“Sin Confirmar”, “Confirmado”) y el vencimiento de su SLA para un evento particular.
- **Cotización de Incremento:** Entidad inmutable interna del Evento emitida ante aumentos de invitados o cambios de alcance. Al autorizarse emite CotizacionIncrementoAprobada.
- **Log de Auditoría:** Registro inmutable estructurado (Append-Only) alimentado por el evento de dominio CambioRegistrado (FR-17).
- **Cuenta de Pagos:** Aggregate Root financiero independiente generado tras la formalización del contrato. Calcula saldos, devoluciones y penalizaciones.
- **Monto Contractual:** Value Object inmutable que calcula el estado financiero: Saldo Pendiente = (Subtotal + Incrementos Aprobados) - (Anticipo + Pagos Realizados).

3.2 Clasificación de entidades y Value Objects

- **Cliente (Aggregate Root):** Identidad única (UUID) y datos de contacto (BR-08, BR-11).
- **Proveedor (Aggregate Root):** Identidad única en catálogo maestro; transiciona entre “Sin Confirmar” y “Confirmado” (BR-10).
- **Evento (Aggregate Root):** Raíz operativa que orquesta asignaciones, cotizaciones de incremento y ciclo de vida contractual.
- **Cuenta de Pagos (Aggregate Root):** Raíz financiera que encapsula cobros, pagos y montos contractuales (BR-09, BR-14).
- **Monto Contractual (Value Object):** Inmutable; calcula saldos sin poseer identidad propia.
- **Gastos Comprometidos (Value Object):** Gastos incurridos por proveedores deducibles del anticipo ante cancelación (BR-14).
- **Confirmación Final de Invitados (Value Object):** Parámetro numérico que fuerza reevaluación si cambia fuera de plazo (BR-05).
- **SLA de Confirmación (Value Object):** Límite temporal estricto de 48 horas calculado a partir de la solicitud al proveedor (BR-04).
- **Log de Auditoría (Entidad Append-Only):** Registro transaccional inmutable alimentado por eventos (FR-17).

3.3 Fichas técnicas de Aggregates

Aggregate Root operativo: Evento

Elemento	Descripción
Raíz	Entidad Evento.
Componentes encapsulados	AsignacionProveedor (1..15), CotizacionIncremento (0..*).
Responsabilidades	Orquestar transiciones de estado contractuales (BR-07), evaluar ventanas de SLA de proveedores (BR-04), calcular límite de 7 días naturales para cambios (BR-02) y emitir eventos de cambio de alcance.
Invariantes duras	1. Nunca formalizar sin cumplir las 4 condiciones (propuesta aceptada, datos completos, servicios confirmados y anticipo cubierto) (BR-07). 2. Máximo 15 asignaciones de proveedores activas simultáneas (BR-15). 3. Cambios solicitados a menos de 7 días naturales requieren confirmación de viabilidad operativa previa (BR-02, BR-12).
Domain Events emitidos	ContratoFormalizado (BR-07), CambioRegistrado (FR-17), AsignacionProveedorVencida (BR-04), AlcanceModificado, CotizacionIncrementoAprobada.

Aggregate Root financiero: Cuenta de Pagos

Elemento	Descripción
Raíz	Entidad CuentaDePagos.
Componentes encapsulados	MontoContractual, GastosComprometidos.
Responsabilidades	Calcular saldos pendientes, registrar pagos, computar penalizaciones y calcular devoluciones de anticipo.
Invariantes duras	1. Los pagos registrados no pueden superar el monto contractual total vigente. 2. La devolución ante cancelación se calcula estrictamente deduciendo los gastos comprometidos pactados (BR-14).
Domain Events emitidos	SaldoContractualLiquidado (BR-09), AnticipoRequerido, DevolucionCalculada.

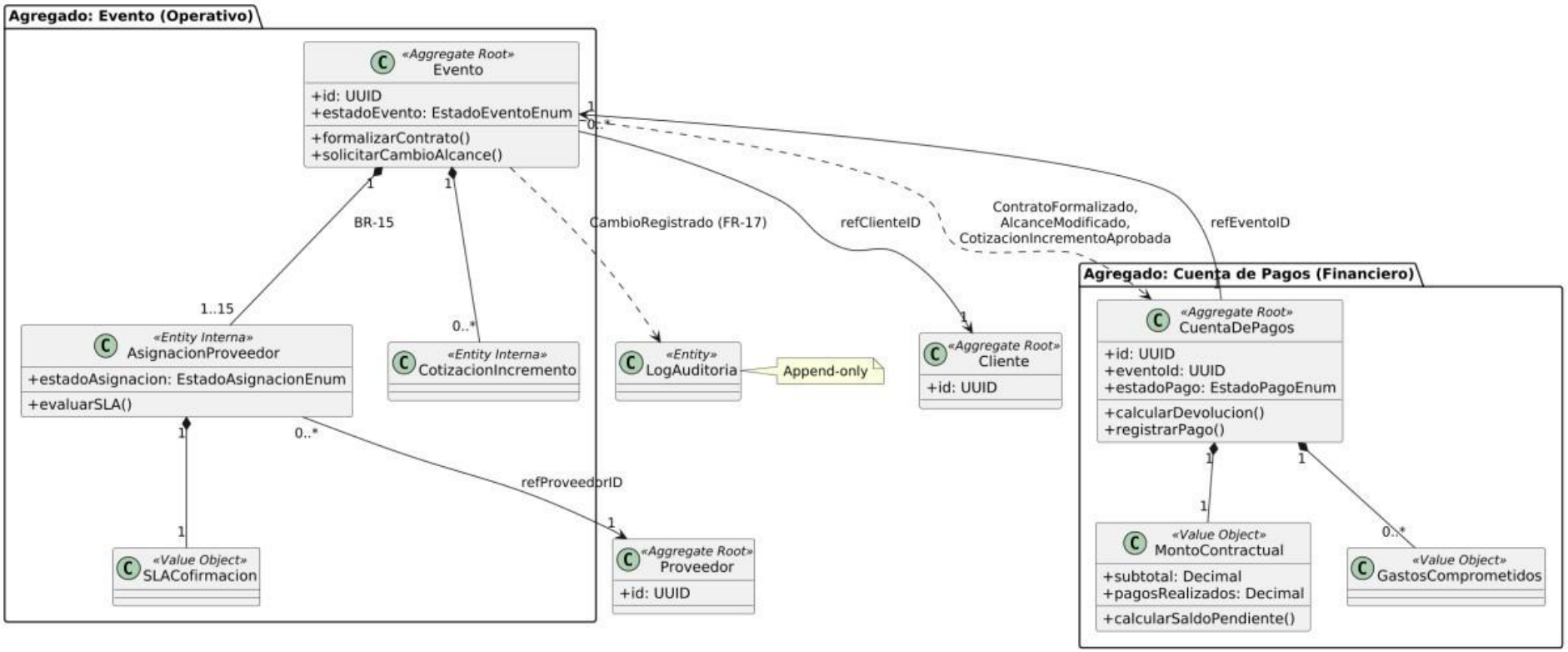


Figura del Mapa de Dominio: agregados Evento (operativo) y Cuenta de Pagos (financiero).

3.4 Decisiones descartadas del mapa de dominio

1. **Consistencia transaccional fuerte entre Evento y Cuenta de Pagos:** Descartada. Se adopta consistencia eventual asíncrona mediante Domain Events para desacoplar el ciclo operativo del financiero.
2. **Evento almacena ID de Cuenta de Pagos:** Descartada. La cuenta nace a partir del contrato; por tanto, CuentaDePagos conserva la referencia eventoId.
3. **Cotización de incremento en el agregado financiero:** Descartada. La negociación de alcance es puramente operativa y reside en Evento; al autorizarse, emite un evento para actualizar el saldo.
4. **Historial de cambios como entidad dentro de Evento:** Descartada. La auditoría se desacopla en una tabla append-only alimentada exclusivamente por Domain Events (FR-17, ADR-003).

4. Service Layer (Etapa 4)

4.1 Propósito de esta etapa

Cada Application Service expone una intención completa: solicitar una cotización, confirmar la respuesta de un proveedor, formalizar un contrato, procesar un cambio de alcance o liquidar un saldo. El servicio recibe la solicitud, obtiene el agregado por su repositorio, invoca el comportamiento del dominio, pide la persistencia y devuelve un resultado. Nada más.

La decisión de Domain Model (ADR-001) trae un riesgo que el propio ADR nombra: un modelo anémico si la lógica se escapa a los servicios. Por eso cada tabla de este capítulo tiene una columna de reglas delegadas, y las reglas BR-16 y BR-17, que son de rol y no de negocio, se resuelven antes de llegar al servicio (ADR-002).

4.2 Casos de uso que orquesta la capa de aplicación

Resumen de la Etapa 1 (Doc., Arquitectura_Fase1.pdf). La tabla completa pertenece al capítulo 1.

ID	Caso de uso	Actor	Reglas	Resultado esperado
CU-01	Solicitar cotización de evento	Cliente / Coordinador	BR-01, BR-05, BR-15	Cotización inicial generada; o alerta por anticipación insuficiente; o evaluación especial.
CU-02	Confirmar respuesta de proveedor	Proveedor / Coordinador	BR-04, BR-10, BR-17	Proveedor confirmado; marcado como “Sin confirmar” por vencer el SLA; o reasignado por el Coordinador.
CU-03	Formalizar contrato de evento	Coordinador	BR-07	Contrato formalizado y monto contractual fijado; o bloqueo por condición faltante.
CU-04	Procesar cambio de alcance	Cliente / Coordinador	BR-02, BR-06, BR-08, BR-12	Cambio aplicado; nueva cotización en espera de aceptación; o rechazo como “No garantizado”.
CU-05	Liquidar saldo o autorizar excepción	Cliente / Administración	BR-09, BR-16	Evento en estado “Pagado”; cancelación contractual por mora; o excepción de pago auditada.

4.3 Decisiones de diseño autorizadas

Estas decisiones sostienen todo el capítulo. Las marcas “Aut.” significan que el equipo las autorizó. Cuando una decisión toca fichas de Rol 1, el capítulo 10.1 lo anota.

ID	Decisión
D1	Los eventos de dominio se despachan en memoria, dentro del proceso, después de confirmar la transacción del agregado origen. Se documenta como ADR-004 (propuesta). Riesgo conocido: si el proceso cae entre el commit y el despacho, el evento se pierde.

ID	Decisión
D2	Commands: FormalizarContratoCommand (CU-03) y ProcesarCambioAlcanceCommand (CU-04). Queries: ConsultarSaldoPendienteQuery (CU-05) y ConsultarAsignacionesProveedorQuery (CU-02), con proyección directa a ViewModels planos.
D3	Diagramas de secuencia: CU-03 con formalización exitosa y CU-04 con cambio fuera de plazo rechazado como “No garantizado”.
D4	Result<T> para desenlaces de negocio esperados y excepciones de dominio tipadas para invariantes. El control de acceso se intercepta antes de la Service Layer (ADR-002).
D5	Una transacción por agregado. userId y rol salen de la sesión en la Service Layer y viajan como parámetros hasta el evento CambioRegistrado.
D6	Un Scheduler externo invoca VerificarVencimientoSLAService (BR-04) y VerificarSaldosVencidosService (BR-09).
D7	CuentaDePagos emite SaldoContractualLiquidado y un handler carga Evento y ejecuta marcarComoPagado() en su propia transacción.
D8	BR-02 en CU-04: el plazo se cumple si fechaEvento - fechaSolicitud >= 7 días.
D9	FormalizarContratoCommand lleva montoAnticipoRecibido y comprobantePagoId. Evento valida que el monto cubra el anticipo pactado como cuarta condición de BR-07.
D10	Payload de ContratoFormalizado: eventoId, clienteId, subtotal, anticipoPactado, anticipoPagado, userId, rol, fechaFormalizacion.
D11	CU-01: SolicitudCotizacion es un DTO de entrada. Evento nace en estado “Solicitud Preliminar” por su Factory, que valida BR-01 y BR-05. Más de 300 invitados pasa a “Evaluación especial de capacidad”. Menos de 20 lanza RangoInvitadosInvalidoException.
D12	CU-04: el Coordinador manda viabilidadConfirmada (booleano) en el comando. Con menos de 7 días y sin viabilidad, Evento rechaza el cambio como “No garantizado” (BR-12). Con viabilidad, genera la CotizacionIncremento en espera de aceptación (BR-08). Las cuatro condiciones de BR-07 son aceptación formal, información completa, servicios confirmados y anticipo pagado, y sustituyen la redacción informal de la ficha de Rol 1.

4.4 Recorrido común de un servicio de escritura

1. El interceptor de roles comprueba que el actor puede ejecutar la operación (ADR-002). Si no puede, el servicio nunca se invoca.
2. La Service Layer arma el comando con userId y rol tomados de la sesión.
3. El servicio carga el agregado con la interfaz de su repositorio.
4. Llama a un método del dominio. Ahí se aplican las reglas y el agregado registra sus eventos.
5. El servicio extrae los eventos, guarda el agregado y confirma la transacción.
6. Despacha los eventos en memoria. Cada reacción corre después, en su propia transacción.
7. Traduce lo que devolvió el dominio a Result<T> y lo entrega a quien lo invocó.

4.5 Tabla oficial de Application Services (CU-01 a CU-05)

Cada Application Service se presenta como una ficha con los mismos seis campos de la tabla oficial: servicio, método, entrada, colaboradores, salida y reglas delegadas al dominio.

S-01 · SolicitudCotizacionService	
Método / caso de uso	solicitarCotizacionDeEvento (CU-01)
Entrada (DTO)	SolicitudCotizacionDTO: clienteId, fechaEvento, cantidadInvitados, fechaSolicitud, userId, rol
Colaboradores / interfaces	IEventoRepository, IUnitOfWork, IDespachadorEventos
Salida	Result<ResultadoSolicitudCotizacion>: eventoId, estadoEvento, estadoCotizacion
Reglas delegadas al dominio	BR-01 (15 días) y BR-05 (20 a 300 invitados) en la Factory de Evento. BR-15 la protege Evento al asignar proveedores, no interviene aquí.

S-02 · ConfirmacionProveedorService	
Método / caso de uso	confirmarRespuestaDeProveedor y sustituirProveedor (CU-02)
Entrada (DTO)	ConfirmarRespuestaProveedorDTO: eventoId, asignacionId, fechaRespuesta, userId, rol. SustituirProveedorDTO: eventoId, asignacionId, nuevoProveedorId, userId, rol
Colaboradores / interfaces	IEventoRepository, IUnitOfWork, IDespachadorEventos
Salida	Result<ResultadoAsignacion>: asignacionId, estadoAsignacion
Reglas delegadas al dominio	BR-04 (SLA de 48 h), BR-10 (sustituto) y BR-15 (máximo de asignaciones activas) en Evento. BR-17 (solo Coordinación sustituye) lo comprueba el interceptor, no el servicio.

S-03 · FormalizarContratoHandler	
Método / caso de uso	formalizarContrato (CU-03)
Entrada (DTO)	FormalizarContratoCommand: eventoId, montoAnticipoRecibido, comprobantePagoId, userId, rol
Colaboradores / interfaces	IEventoRepository, IUnitOfWork, IDespachadorEventos
Salida	Result<ResultadoFormalizacion>: eventoId, estadoEvento, fechaFormalizacion
Reglas delegadas al dominio	BR-07: las cuatro condiciones, incluido que el monto recibido cubra el anticipo pactado, las comprueba Evento.formalizarContrato.

S-04 · ProcesarCambioAlcanceHandler	
Método / caso de uso	procesarCambioAlcance (CU-04)
Entrada (DTO)	ProcesarCambioAlcanceCommand: eventoId, cambio, fechaSolicitud, viabilidadConfirmada, userId, rol
Colaboradores / interfaces	IEventoRepository, IUnitOfWork, IDespachadorEventos
Salida	Result<ResultadoCambioAlcance>: eventoId, resultado, cotizacionIncrementoId
Reglas delegadas al dominio	BR-02 (7 días), BR-12 (fuera de plazo), BR-06 (dos cambios gratuitos) y BR-08 (costo) en Evento.solicitarCambioAlcance.

S-05 · LiquidacionSaldoService	
Método / caso de uso	liquidarSaldo y autorizarExcepcionDePago (CU-05)
Entrada (DTO)	LiquidarSaldoDTO: eventoId, monto, fechaPago, userId, rol. AutorizarExcepcionPagoDTO: eventoId, motivo, userId, rol
Colaboradores / interfaces	ICuentaDePagosRepository, IUnitOfWork, IDespachadorEventos. Para la excepción, el agregado dueño está pendiente (capítulo 10.2).
Salida	Result<ResultadoLiquidacion>: eventoId, saldoPendiente, estadoPago
Reglas delegadas al dominio	Saldo y tope de pagos en CuentaDePagos. BR-09 (liquidar tres días antes) la evalúa el dominio. BR-16 (solo Administración autoriza excepciones) lo comprueba el interceptor.

4.6 Detalle de cada operación

S-01	
Agregado que interviene	Evento (Factory; el agregado todavía no existe en la base).
Errores y desenlaces alternativos	Con menos de 20 invitados: RangoInvitadosInvalidoException, que el servicio convierte en Result.fallo con la regla BR-05. Con menos de 15 días: desenlace “anticipación insuficiente” (Doc., resultado de CU-01). Con más de 300 invitados: desenlace “Evaluación especial de capacidad” (Aut.).
Límite transaccional	Una transacción sobre el Evento nuevo. Empieza al llamar la Factory y termina en confirmar(). Con anticipación insuficiente no se abre transacción: supuesto Prop., el Evento no se persiste.
Eventos de dominio	Ninguno documentado para CU-01.

S-02	
Agregado que interviene	Evento y sus AsignacionProveedor.
Errores y desenlaces alternativos	Respuesta fuera de SLA y reglas de sustitución: sin definir en las fuentes (capítulo 10.2). Asignación inexistente: Result.fallo con código de aplicación.
Límite transaccional	Una transacción sobre Evento. Empieza al cargarlo y termina en confirmar().
Eventos de dominio	Ninguno documentado para estas dos operaciones. AsignacionProveedorVencida sale por P-01, no por aquí.

S-03	
Agregado que interviene	Evento.
Errores y desenlaces alternativos	Falta una condición de BR-07: CondicionFormalizacionFaltanteException, que indica cuál falta. El servicio la traduce a Result.fallo. Evento no encontrado: Result.fallo.
Límite transaccional	Una transacción sobre Evento: cargar, formalizar, guardar y confirmar. CuentaDePagos se crea fuera, en H-01.
Eventos de dominio	ContratoFormalizado, CambioRegistrado.

S-04	
Agregado que interviene	Evento (con CotizacionIncremento).
Errores y desenlaces alternativos	“No garantizado” y “cotización pendiente de aceptación” son desenlaces previstos y vuelven como Result.exito. Evento no encontrado: Result.fallo.
Límite transaccional	Una transacción sobre Evento: cargar, solicitar el cambio, guardar y confirmar.
Eventos de dominio	CambioRegistrado (supuesto Prop. para el rechazo). AlcanceModificado y CotizacionIncrementoAprobada solo salen cuando el cambio se aplica o la cotización se aprueba.

S-05	
Agregado que interviene	CuentaDePagos.
Errores y desenlaces alternativos	Pago que supere el monto contractual: PagoExcedeMontoContractualException (invariante 1 del mapa). Cuenta aún no creada: Result.fallo.
Límite transaccional	Una transacción sobre CuentaDePagos. Marcar el Evento como “Pagado” ocurre después, en H-02, con otra transacción.
Eventos de dominio	SaldoContractualLiquidado cuando el saldo llega a cero.

4.7 Servicios complementarios: Scheduler y manejadores de eventos

P-01 · VerificarVencimientoSLAService.verificarVencimientoSLA(ahora)	
Qué lo dispara	Scheduler externo, periódico (D6).
Qué hace	Busca eventos con asignaciones pendientes y le pide a cada uno que evalúe su SLA.
Colaboradores	IEventoRepository.listarConAsignacionesPendientes(), IUnitOfWork, IDespachadorEventos
Reglas delegadas	BR-04 en Evento y AsignacionProveedor.evaluarSLA.
Transacción	Una por evento, para que un fallo no frene a los demás (Prop.).

P-02 · VerificarSaldosVencidosService.verificarSaldosVencidos(ahora)	
Qué lo dispara	Scheduler externo, periódico (D6).
Qué hace	Busca eventos formalizados que no están en “Pagado” y le pide a cada uno que evalúe su liquidación.
Colaboradores	IEventoRepository.listarFormalizadosSinLiquidar(), IUnitOfWork, IDespachadorEventos
Reglas delegadas	BR-09 en Evento.evaluarLiquidacion(ahora) (Prop., método nuevo). La consecuencia se llama “cancelación contractual por mora”, igual que en Fase 1.
Transacción	Una por evento (Prop.).

H-01 · ManejadorContratoFormalizado	
Qué lo dispara	ContratoFormalizado
Qué hace	Crea la CuentaDePagos con subtotal, anticipo pactado y anticipo ya pagado. Si ya existe una cuenta para ese eventoId, no crea otra (Prop.).
Colaboradores	ICuentaDePagosRepository, IUnitOfWork
Reglas delegadas	Creación y cálculo inicial en CuentaDePagos.
Transacción	Una, sobre CuentaDePagos.

H-02 · ManejadorSaldoContractualLiquidado	
Qué lo dispara	SaldoContractualLiquidado
Qué hace	Carga el Evento por el eventoId de la cuenta y ejecuta marcarComoPagado() (D7).
Colaboradores	IEventoRepository, IUnitOfWork
Reglas delegadas	La transición a “Pagado” la valida Evento.
Transacción	Una, sobre Evento.

H-03 · ManejadorCambioRegistrado	
Qué lo dispara	CambioRegistrado
Qué hace	Agrega un registro a la auditoría con userId, rol, tipo de cambio y fecha. Nunca modifica registros anteriores.
Colaboradores	IRegistroAuditoriaRepository, IUnitOfWork
Reglas delegadas	Ninguna: solo escribe lo que trae el evento (ADR-003, FR-17).
Transacción	Una, sobre la tabla de auditoría.

H-04 · ManejadorCotizacionIncrementoAprobada	
Qué lo dispara	CotizacionIncrementoAprobada
Qué hace	Actualiza el Monto Contractual de la cuenta con el incremento aprobado.
Colaboradores	ICuentaDePagosRepository, IUnitOfWork
Reglas delegadas	Recálculo en CuentaDePagos. Prop.: el payload del evento y el método de la cuenta no están definidos.
Transacción	Una, sobre CuentaDePagos.

4.8 Interfaces que usa la capa de aplicación

Interfaz	Métodos que usa la capa de aplicación	Quién la usa
IEventoRepository	obtenerPorId(eventoId), guardar(evento), listarConAsignacionesPendientes(), listarFormalizadosSinLiquidar()	S-01 a S-04, P-01, P-02, H-02
ICuentaDePagosRepository	obtenerPorEventoId(eventoId), guardar(cuenta)	S-05, H-01, H-04
IRegistroAuditoriaRepository	agregar(registro). No tiene métodos para modificar ni borrar.	H-03
IUnitOfWork	confirmar(), revertir()	Todos los que escriben
IDespachadorEventos	publicar(eventos)	S-01 a S-05, P-01, P-02

El control de acceso por roles es un interceptor delante de la Service Layer (ADR-002). Por eso ningún servicio de este capítulo verifica el rol.

4.9 Estrategia de errores y resultados

La decisión D4 separa lo que el negocio espera de lo que el negocio prohíbe y de lo que simplemente falla. Cada categoría tiene un tratamiento distinto.

Situación	Ejemplo	Cómo se representa
Desenlace de negocio previsto en el caso de uso	“No garantizado” (CU-04), “Evaluación especial de capacidad” (CU-01)	Result.exito con el desenlace dentro del valor devuelto. El estado se persiste.
Violación de una invariante del dominio	Falta una condición de BR-07; menos de 20 invitados; pago mayor al monto contractual	El dominio lanza una excepción tipada. El handler la captura y devuelve Result.fallo. No cambia nada en el agregado.
Rol no autorizado	Un Cliente intenta autorizar una excepción de pago (BR-16)	Lo responde el interceptor antes de llegar al servicio (ADR-002).
Fallo técnico	Base de datos caída, conflicto de concurrencia	Excepción técnica que se propaga. No se traduce a Result.

4.10 Por qué los servicios no absorben las reglas

Cada regla tiene un dueño y el servicio solo hace de mensajero.

Regla	Dónde vive	Qué hace el servicio
BR-01 y BR-05	Factory de Evento	Pasa fechas e invitados y traduce el resultado.
BR-15	Invariante de Evento	Nada.
BR-04	Evento y AsignacionProveedor.evaluarSLA	P-01 solo entrega la hora actual (ahora).
BR-10	Evento (detalle pendiente)	Pasa el proveedor sustituto.

Regla	Dónde vive	Qué hace el servicio
BR-07	Evento.formalizarContrato	Pasa el monto y el comprobante. No cuenta condiciones.
BR-02, BR-12, BR-06, BR-08	Evento.solicitarCambioAlcance	Pasa la fecha de solicitud y la viabilidad. No resta días.
BR-09	Evento.evaluarLiquidacion (Prop.) y CuentaDePagos	P-02 solo entrega ahora.
Tope de pagos y saldo	CuentaDePagos y MontoContractual	Pasa el monto del pago.
BR-16 y BR-17	Interceptor de roles (ADR-002)	Nada. El dominio tampoco conoce al usuario.

4.11 Revisión obligatoria de la Etapa 4

La guía exige responder cinco preguntas sobre la Service Layer. Las respuestas no son todas un sí. Donde hay un riesgo, se dice.

Pregunta	Evidencia del diseño	Resultado
1. ¿El nombre del método expresa una acción del negocio?	solicitarCotizacionDeEvento, confirmarRespuestaDeProveedor, formalizarContrato, procesarCambioAlcance, liquidarSaldo.	Cumple con una observación. Los handlers de CQRS exponen un método genérico manejar. El nombre del negocio queda en la clase y en el comando, y así se dibuja en los diagramas.
2. ¿El método coordina o contiene decisiones que deberían estar en el dominio?	S-03 no cuenta condiciones. S-04 no resta días. Los dos solo cargan, llaman al agregado, guardan y traducen.	Cumple en los servicios de escritura. Queda un riesgo en ConsultarSaldoPendienteHandler, que repite la fórmula del saldo en el lado de lectura (D2). Se acepta, pero hay que mantener ambas copias iguales.

Pregunta	Evidencia del diseño	Resultado
3. ¿El servicio depende de interfaces o de detalles concretos de base de datos?	Todos usan las cinco interfaces de la sección 4.8. Ningún servicio nombra tablas ni SQL.	Cumple, con interfaces provisionales hasta la Etapa 5. IEventoRepository ya tiene cuatro métodos, dos de ellos solo para el Scheduler, y podría partirse si crece.
4. ¿La operación tiene un inicio y un final transaccional comprensible?	Una transacción por agregado, desde la carga hasta confirmar(). Los eventos se despachan después. Los diagramas la marcan con fragmentos “Transacción 1, 2, 3”.	Cumple, con dos riesgos. El despacho después del commit puede perder un evento si el proceso cae (ADR-004). Y si H-02 falla después de liquidarSaldo, el Evento queda sin “Pagado” y las fuentes no definen política de reintento. No se inventó una.
5. ¿Los errores del dominio pueden explicarse con términos del negocio?	CondicionFormalizacionFaltanteException (BR-07), RangoInvitadosInvalidoException (BR-05), PagoExcedeMontoContractualException, “No garantizado” (BR-12).	Cumple para los casos con reglas definidas. Falta el catálogo de errores de BR-10 y de la excepción de pago, porque las fuentes no los describen.

5. Repository y Unit of Work (Etapa 5)

5.1 Definición de interfaces del patrón Repository

En el marco de Clean Architecture y Domain-Driven Design, la infraestructura de persistencia se abstrae a través del patrón Repository. Estas interfaces simulan colecciones en memoria de Aggregate Roots, expresando exclusivamente las necesidades de consulta y almacenamiento del negocio sin acoplarse a Entity Framework Core ni sentencias SQL.

Tabla oficial de repositorios

IEventoRepository	
Necesidad del dominio	Cargar y persistir el Aggregate Root Evento asegurando límites de proveedores y estados operativos.
Métodos propuestos	obtenerPorId(eventoId: UUID): Task<Evento?> guardar(evento: Evento): Task listarConAsignacionesPendientes(): Task<IEnumerable<Evento>> listarFormalizadosSinLiquidar(): Task<IEnumerable<Evento>>
Objeto que recupera o guarda	Agregado Evento

ICuentaDePagosRepository	
Necesidad del dominio	Gestionar la persistencia del agregado financiero independiente y control de saldos contractuales.
Métodos propuestos	obtenerPorEventoId(eventoId: UUID): Task<CuentaDePagos?> guardar(cuenta: CuentaDePagos): Task
Objeto que recupera o guarda	Agregado CuentaDePagos

IRegistroAuditoriaRepository	
Necesidad del dominio	Registro append-only inmutable de acciones de negocio para trazabilidad (FR-17, ADR-003).
Métodos propuestos	agregar(registro: LogAuditoria): Task (Sin Update ni Delete)
Objeto que recupera o guarda	Entidad LogAuditoria

IProveedorRepository	
Necesidad del dominio	Consulta y validación de disponibilidad y sustitutos en el catálogo maestro.
Métodos propuestos	obtenerPorId(proveedorId: UUID): Task<Proveedor?> listarPorCategoria(categoria: String): Task<IEnumerable<Proveedor>>
Objeto que recupera o guarda	Agregado Proveedor

5.2 Alcance transaccional y Unit of Work (IUnitOfWork)

El Unit of Work coordina las operaciones transaccionales garantizando atomicidad (ACID). En estricto apego al Mapa de Dominio y al ADR-004:

- **Alcance por transacción:** Cada unidad de trabajo cubre exclusivamente a UN solo Aggregate Root. Evento y CuentaDePagos jamás comparten una transacción atómica conjunta.
- **Ciclo de vida:** La transacción inicia al cargar el agregado desde el repositorio y concluye con confirmar() (CommitAsync). Ante cualquier excepción, se invoca revertir() (RollbackAsync).
- **Despacho de eventos:** Los eventos de dominio acumulados en memoria solo se despachan tras la confirmación exitosa de la transacción en la base de datos (ADR-004).

5.3 Contraste con los requisitos de la capa de aplicación

Al integrar este capítulo se revisó que cubra lo que la capa de aplicación (capítulo 4) necesita de la persistencia. La mayoría de los requisitos se cumplen. Quedan tres puntos abiertos.

Requisito de la capa de aplicación	Cómo lo cubre este capítulo	Estado
Cada agregado se guarda con su propia transacción; Evento y CuentaDePagos nunca se confirman juntos.	Sección 5.2, “Alcance por transacción”.	Cubierto
IUnitOfWork permite confirmar() y revertir() sobre una transacción abierta al cargar el agregado.	Sección 5.2, “Ciclo de vida”, y la interfaz IUnitOfWork del Anexo A.	Cubierto
El repositorio de auditoría solo expone agregar.	Sección 5.1: IRegistroAuditoriaRepository, “Sin Update ni Delete”.	Cubierto
IEventoRepository ofrece las dos consultas que necesita el Scheduler.	Sección 5.1 las lista: listarConAsignacionesPendientes y listarFormalizadosSinLiquidar.	Cubierto
ICuentaDePagosRepository busca por eventoId.	Sección 5.1: obtenerPorEventoId.	Cubierto
Existe IDespachadorEventos para entregar los eventos después del commit.	La sección 5.2 describe el despacho tras la confirmación, pero la interfaz no aparece en 5.1 ni en el Anexo A.	Pendiente: nombrar la interfaz
Existen interfaces de lectura para CQRS (IConsultaSaldoPendiente, IConsultaAsignacionesProveedor).	No aparecen en 5.1 ni en el Anexo A.	Pendiente: definirlas
Se define qué pasa si confirmar() falla o si un manejador de eventos falla.	La sección 5.2 indica revertir() ante una excepción. No hay política de reintento para los manejadores de eventos.	Parcial

Requisito de la capa de aplicación	Cómo lo cubre este capítulo	Estado
IProveedorRepository (nuevo en este capítulo).	Ningún servicio del capítulo 4 lo usa. S-02 podría necesitarlo para validar al proveedor sustituto (BR-10).	A confirmar

6. CQRS básico (Etapa 6)

6.1 Cómo se separan escritura y lectura

Un Command expresa la intención de cambiar el estado del sistema. Pasa por un handler, que carga el agregado por su repositorio, llama a un método del dominio, guarda y confirma. Ahí se aplican las reglas.

Una Query solo lee. Su handler no carga agregados ni llama a métodos del dominio: le pide los datos a una interfaz de lectura y los devuelve en un ViewModel plano. Nunca modifica nada.

Esta separación es lógica. Las dos mitades usan el mismo almacén de datos (sección 6.6).

6.2 Tabla oficial de Commands y Queries

Command · FormalizarContratoCommand	
Datos de entrada	eventoId, montoAnticipoRecibido, comprobantePagoId, userId, rol
Handler	FormalizarContratoHandler
Resultado (DTO / ViewModel)	Result<ResultadoFormalizacion>
Justificación	CU-03 cambia el estado de Evento y dispara la creación de la cuenta financiera. Concentra una regla fuerte (BR-07) que debe vivir en el dominio.

Command · ProcesarCambioAlcanceCommand	
Datos de entrada	eventoId, cambio, fechaSolicitud, viabilidadConfirmada, userId, rol
Handler	ProcesarCambioAlcanceHandler
Resultado (DTO / ViewModel)	Result<ResultadoCambioAlcance>
Justificación	CU-04 tiene tres desenlaces posibles y varias reglas de tiempo y costo (BR-02, BR-06, BR-08, BR-12). Es el caso donde más se nota que el servicio no decide.

Query · ConsultarSaldoPendienteQuery	
Datos de entrada	eventoId
Handler	ConsultarSaldoPendienteHandler
Resultado (DTO / ViewModel)	Result<SaldoPendienteViewModel>
Justificación	CU-05 necesita que el Cliente y Administración vean cuánto falta pagar antes de liquidar (BR-09). Es una lectura sobre CuentaDePagos.

Query · ConsultarAsignacionesProveedorQuery	
Datos de entrada	eventoId
Handler	ConsultarAsignacionesProveedorHandler
Resultado (DTO / ViewModel)	Result<List<AsignacionProveedorDTO>>
Justificación	CU-02 necesita que el Coordinador vea qué proveedores están “Sin confirmar” y hasta cuándo corre su SLA (BR-04). Es una lectura sobre Evento.

6.3 Dependencias y relación con el resto del diseño

C-01 · Command	
Servicio de aplicación	S-03 (el handler es el servicio)
Componentes de dominio e interfaces	IEventoRepository, Evento.formalizarContrato, IUnitOfWork, IDespachadorEventos
Efecto o datos consultados	Escribe el Evento y emite ContratoFormalizado y CambioRegistrado.
Reglas relacionadas	BR-07

C-02 · Command	
Servicio de aplicación	S-04 (el handler es el servicio)
Componentes de dominio e interfaces	IEventoRepository, Evento.solicitarCambioAlcance, IUnitOfWork, IDespachadorEventos
Efecto o datos consultados	Escribe el Evento (cambio rechazado, aplicado o cotización en espera) y emite CambioRegistrado.
Reglas relacionadas	BR-02, BR-06, BR-08, BR-12

Q-01 · Query	
Servicio de aplicación	Ninguno. Lo invoca directamente la capa de entrada.
Componentes de dominio e interfaces	IConsultaSaldoPendiente
Efecto o datos consultados	Lee subtotal, incrementos aprobados, anticipo pactado, pagos realizados y estado de pago de la cuenta del evento.
Reglas relacionadas	BR-09 y fórmula del saldo (Doc.)

Q-02 · Query	
Servicio de aplicación	Ninguno. Lo invoca directamente la capa de entrada.
Componentes de dominio e interfaces	IConsultaAsignacionesProveedor
Efecto o datos consultados	Lee las asignaciones del evento con su estado y fecha límite de confirmación.
Reglas relacionadas	BR-04

6.4 Contratos de DTOs y ViewModels

Ningún DTO de esta sección es una entidad del dominio. Los de lectura tampoco se reutilizan como modelo de escritura.

Envoltorio de resultado

Result<T> (decisión D4)

Campo	Tipo	Descripción
esExito	boolean	Verdadero cuando hay un desenlace de negocio, incluso uno alternativo como “No garantizado”.
valor	T	Presente solo si esExito es verdadero.
error	ErrorNegocio	Presente solo si esExito es falso.

ErrorNegocio

Campo	Tipo	Descripción
codigo	String	Código estable. Ejemplos: CONDICION_FORMALIZACION_FALTANTE, RANGO_INVITADOS_INVALIDO, PAGO_EXCEDE_MONTO_CONTRACTUAL, EVENTO_NO_ENCONTRADO, CUENTA_NO_DISPONIBLE.
mensaje	String	Texto en términos del negocio.
reglaRelacionada	String	Identificador de la regla, por ejemplo BR-07. Puede venir vacío.

El rechazo por falta de permisos no lo produce el handler. Lo responde el interceptor de roles antes de llegar a la Service Layer (ADR-002).

Commands y su salida

FormalizarContratoCommand

Campo	Tipo	Origen
eventoId	UUID	Lo envía el Coordinador.
montoAnticipoRecibido	Decimal	Lo envía el Coordinador (D9).
comprobantePagoId	UUID	Lo envía el Coordinador (D9).

Campo	Tipo	Origen
userId	UUID	Lo toma la Service Layer de la sesión (D5).
rol	String	Lo toma la Service Layer de la sesión (D5).

ResultadoFormalizacion: eventoId (UUID), estadoEvento (String), fechaFormalizacion (DateTime).

ProcesarCambioAlcanceCommand

Campo	Tipo	Origen
eventoId	UUID	Lo envía el Coordinador.
cambio	CambioAlcanceDTO	Lo envía el Coordinador.
fechaSolicitud	Date	Fecha en que el Cliente pidió el cambio. Con ella el dominio calcula fechaEvento - fechaSolicitud (D8).
viabilidadConfirmada	boolean	El Coordinador la obtiene de los proveedores (D12).
userId	UUID	De la sesión (D5).
rol	String	De la sesión (D5).

CambioAlcanceDTO (Prop.): descripcion (String) y nuevaConfirmacionInvitados (Integer, opcional, porque el glosario trae “Confirmación Final Invitados” como un número que fuerza reevaluación). El BRD debe confirmar qué más describe un cambio.

ResultadoCambioAlcance

Campo	Tipo	Descripción
eventoId	UUID	
resultado	String	Aplicado, CotizacionPendienteDeAceptacion o NoGarantizado.
cotizacionIncrementoId	UUID	Solo cuando el resultado es CotizacionPendienteDeAceptacion.

Queries y ViewModels de lectura

ConsultarSaldoPendienteQuery: eventoId (UUID).

SaldoPendienteViewModel

Campo	Tipo	Descripción
eventoId	UUID	
subtotal	Decimal	
totalIncrementosAprobados	Decimal	
anticipoPactado	Decimal	
pagosRealizados	Decimal	
saldoPendiente	Decimal	(subtotal + totalIncrementosAprobados) - (anticipoPactado + pagosRealizados), la fórmula documentada.
estadoPago	String	Estado de pago de la cuenta. El mapa define un enumerado EstadoPagoEnum y solo uno de sus valores aparece en las fuentes: “Pagado”.

ConsultarAsignacionesProveedorQuery: eventoId (UUID).

AsignacionProveedorDTO

Campo	Tipo	Descripción
asignacionId	UUID	
proveedorId	UUID	El mapa solo documenta el id del Proveedor. El nombre queda pendiente.
estadoAsignacion	String	“Sin confirmar” o “Confirmado”, según el glosario.
fechaLimiteConfirmacion	DateTime	Límite de 48 h que el dominio ya calculó y guardó (BR-04). La Query no vuelve a calcularlo.

6.5 Recorrido de una Query y consecuencia de la consistencia eventual

En ConsultarSaldoPendienteQuery la capa de entrada manda la Query, ConsultarSaldoPendienteHandler llama a IConsultaSaldoPendiente.obtener(eventoId), la implementación de infraestructura lee y proyecta, y el handler devuelve el ViewModel. No interviene IUnitOfWork, ni un agregado, ni se emite un evento.

Hay una consecuencia que conviene tener presente. La cuenta se crea de forma asíncrona después de ContratoFormalizado (D1). Si alguien consulta el saldo en ese instante, la cuenta todavía puede no existir. El handler devuelve en ese caso Result.fallo con CUENTA_NO_DISPONIBLE, que es un desenlace del diseño y no un error técnico.

6.6 Por qué no se duplica la base de datos física

La guía dice que CQRS básico no exige bases separadas. En Eventos Nova tampoco ayudaría, por estas razones:

- La consistencia eventual ya existe entre Evento y CuentaDePagos (mapa de dominio). Agregar un segundo almacén de lectura sumaría otra sincronización que el negocio no pide.
- Las lecturas elegidas son simples: un saldo y una lista de asignaciones de un solo evento. No hay consultas pesadas ni de reportes que justifiquen otro modelo físico.
- El beneficio que se busca es de diseño: que la lectura devuelva el modelo que necesita la pantalla sin pasar por los agregados. Eso se consigue con handlers e interfaces de lectura distintas sobre el mismo almacén.

El costo es que la fórmula del saldo existe dos veces: en MontoContractual para escritura y en la proyección para lectura (D2). Mientras la proyección copie la fórmula documentada tal cual, no hay conflicto, pero cualquier cambio en ella obliga a tocar las dos.

6.7 Qué partes no necesitan esta separación

- CU-01, CU-02 y la liquidación de CU-05 también escriben, pero no se eligieron para demostrar CQRS. Siguen como servicios de aplicación normales (S-01, S-02, S-05). Podrían convertirse en Commands sin cambiar su lógica.
- P-01, P-02 y los manejadores H-01 a H-04 son reacciones. No tienen un Command ni una Query porque ninguna persona las pide.
- CU-01 no tiene una necesidad de lectura propia entre las cuatro Queries elegidas.

6.8 Control de coherencia

Comprobación	Resultado
Cada Command tiene un efecto de escritura identificable	Cumple. C-01 y C-02 escriben Evento.

Comprobación	Resultado
Cada Query tiene un propósito de lectura definido	Cumple. Q-01 muestra el saldo y Q-02 muestra el SLA de cada asignación.
Las operaciones salen de casos de uso reales	Cumple. CU-03, CU-04, CU-05 y CU-02.
Los handlers tienen dependencias explícitas	Cumple (sección 6.3). Las interfaces de lectura son provisionales.
Las reglas permanecen en el dominio	Cumple en los Commands. Riesgo aceptado en Q-01 por la fórmula duplicada.
Las Queries no modifican el estado	Cumple. No usan IUnitOfWork ni cargan agregados.
Los DTO no son entidades del dominio	Cumple (sección 6.4).
La persistencia es compatible con Repository y Unit of Work	Cumple de forma provisional hasta la Etapa 5.
Los eventos respetan la consistencia eventual	Cumple. Los dos Commands confirman su transacción antes de despachar.
Nombres y parámetros no contradicen las reglas	Cumple. Los parámetros salen de las decisiones D8 a D12.

7. Revisión SOLID (Etapa 7)

7.1 Evaluación formal de los cinco principios

Evaluación formal de los 5 principios de diseño sobre la capa de persistencia e infraestructura:

Principio	Hallazgo / evidencia de riesgo	Corrección y mitigación aplicada
S (Single Responsibility)	Entidades del dominio como Evento o Proveedor corrían el riesgo de acoplar métodos directos de persistencia (ej. GuardarEnBD()).	Se extrajo toda persistencia hacia la infraestructura mediante interfaces Repository. Las entidades solo gobiernan reglas puras de negocio.
O (Open/Closed)	Agregar nuevas entidades requería modificar la lógica de persistencia centralizada del sistema.	Se estructuró un contrato genérico base de repositorio IRepository<T>, abierto a la extensión de interfaces especializadas y cerrado a modificaciones.
L (Liskov Substitution)	Incompatibilidad potencial al sustituir implementaciones de repositorio simuladas en memoria (InMemory) por implementaciones de Entity Framework Core.	Todas las clases de infraestructura implementan estrictamente las firmas de interfaz sin alterar el comportamiento esperado por la capa de aplicación.
I (Interface Segregation)	Riesgo de una interfaz monolítica de datos con múltiples métodos forzando a los servicios a depender de operaciones innecesarias.	Se segregaron contratos por Aggregate Root (IEventoRepository, ICuentaDePagosRepository, IRegistroAuditoriaRepository). Cada servicio consume solo lo que requiere.
D (Dependency Inversion)	La capa de aplicación instanciaba directamente clases concretas de infraestructura (ej. new SqlEventoRepository()).	Los Application Services dependen exclusivamente de abstracciones (interfaces inyectadas mediante el contenedor IoC).

7.2 Observaciones complementarias desde la capa de aplicación

Estas observaciones se redactaron antes de integrar el capítulo anterior y se conservan como contraste. Miran los mismos principios desde los servicios de aplicación y no reemplazan la evaluación formal.

Principio	Componente	Evidencia	Observación	Estado
SRP	S-03 y S-04	Cargan, llaman al dominio, guardan y traducen. No evalúan reglas.	Cada handler tiene una sola razón para cambiar.	Confirma una buena decisión
SRP	S-05 LiquidacionSaldoService	Agrupa liquidarSaldo (Cliente) y autorizarExcepcionDePago (Administración).	Dos actores y dos motivos de cambio en una clase. Podría separarse si crece.	Riesgo, sin corregir
OCP	Manejadores de eventos	Agregar H-04 no obliga a editar S-04 ni a ningún publicador.	Una reacción nueva se suma sin tocar el código existente.	Confirma una buena decisión
ISP	IEventoRepository	Cuatro métodos, dos de ellos solo para el Scheduler.	Los handlers de comandos dependen de métodos que no usan.	Riesgo, sin corregir
DIP	Todos los servicios	Dependen de interfaces (sección 4.8). Ninguno nombra SQL ni un ORM.	La infraestructura se puede sustituir sin tocar la capa de aplicación.	Confirma una buena decisión
LSP	Sin evaluar	No hay jerarquías ni implementaciones sustituibles definidas en el diseño de aplicación.	La evaluación de LSP corresponde a las implementaciones de infraestructura del cuadro anterior.	No evaluable desde esta capa

8. Diagramas de secuencia (Entregable 03)

8.1 Qué muestran y cómo leerlos

Los dos diagramas representan la misma arquitectura que describen los capítulos 4 y 6. Uno cubre un flujo principal (CU-03) y el otro un flujo con una regla de plazo y un desenlace alternativo (CU-04). Los participantes se llaman igual que los componentes de las tablas.

Dos convenciones ayudan a leer los dibujos. Los fragmentos “Transacción 1, 2, 3” encierran lo que se confirma de una sola vez. Las flechas con punta abierta son llamadas asíncronas: el emisor no espera la respuesta.

Participante	Tipo	Responsabilidad	Referencia
Coordinador	Actor	Inicia la operación y recibe el resultado.	CU-03, CU-04
ControlAccesoRBAC	Interceptor	Verifica el rol antes de entrar a la Service Layer. No interviene el dominio.	ADR-002, sección 4.8
FormalizarContratoHandler	Application Service	Coordina CU-03.	S-03, C-01
ProcesarCambioAlcanceHandler	Application Service	Coordina CU-04.	S-04, C-02
IEventoRepository	Interfaz	Carga y guarda el agregado Evento.	Sección 4.8
Evento	Aggregate Root	Aplica las reglas y registra los eventos.	Mapa de dominio
IUnitOfWork	Interfaz (provisional)	Confirma la transacción.	Etapa 5
IDespachadorEventos	Interfaz	Entrega los eventos en memoria después del commit.	D1, ADR-004
ManejadorContratoFormalizado	Event Handler	Crea la CuentaDePagos.	H-01
ICuentaDePagosRepository y CuentaDePagos	Interfaz y Aggregate Root	Guardan la cuenta que nace del contrato.	S-05, H-01
ManejadorCambioRegistrado	Event Handler	Escribe la auditoría.	H-03

Participante	Tipo	Responsabilidad	Referencia
IRegistroAuditoriaRepository	Interfaz	Solo agrega registros.	ADR-003
Base de datos	Almacén	Un solo almacén para escritura y lectura.	Sección 6.6

8.2 Diagrama 1: CU-03 Formalizar contrato de evento (flujo exitoso)

Campo	Contenido
Identificador	Secuencia_CU03_Exito
Caso de uso	CU-03 Formalizar contrato de evento. Actor: Coordinador.
Regla	BR-07, cuatro condiciones obligatorias (D12).
Operación	FormalizarContratoCommand en FormalizarContratoHandler (S-03, C-01).
Objetivo	Mostrar cómo un contrato se formaliza en una transacción sobre Evento y cómo la cuenta de pagos y la auditoría reaccionan después, cada una en la suya.

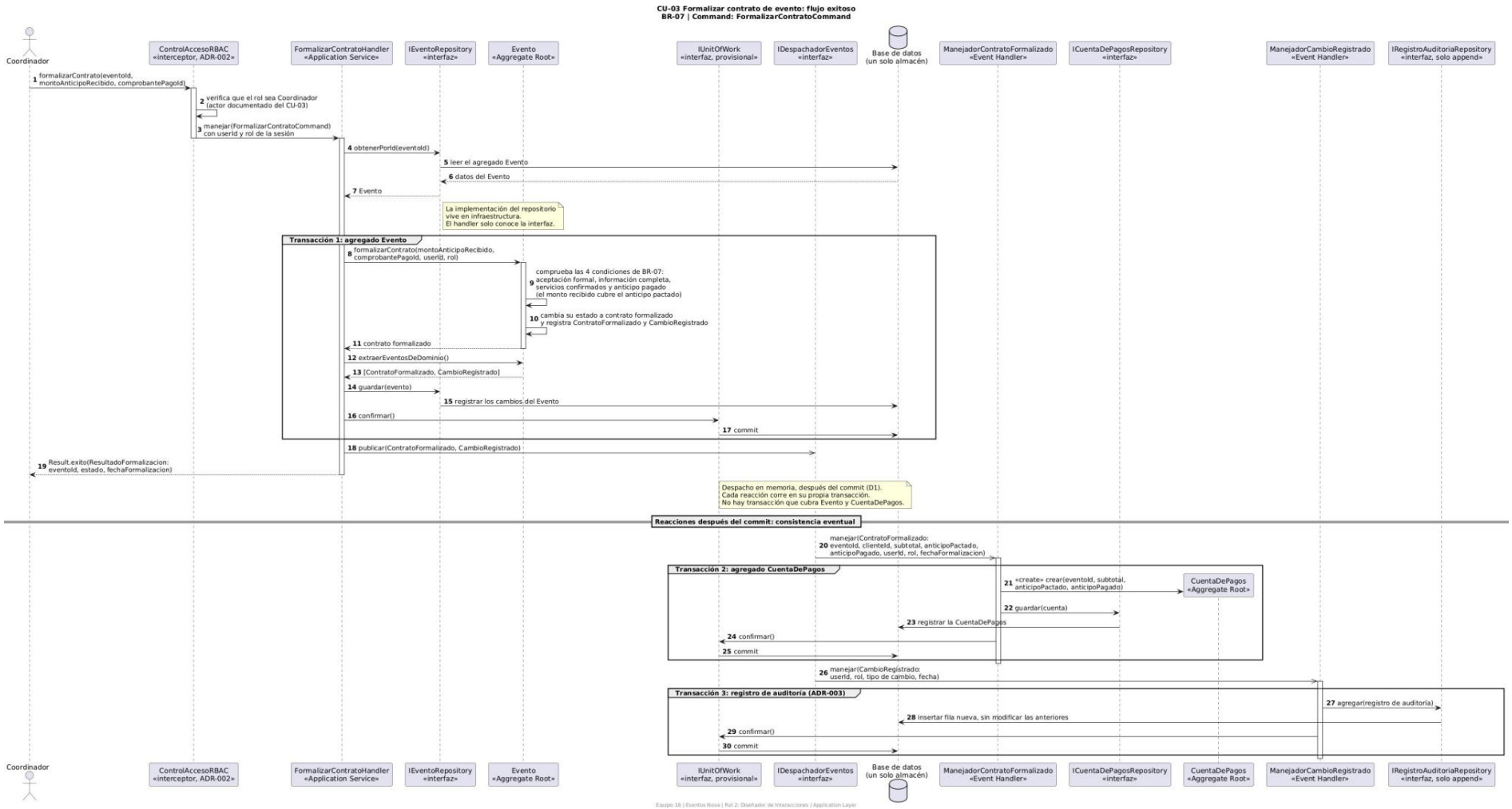


Figura 1: Secuencia CU-03, formalización de contrato exitosa (Secuencia_CU03_Exito).

Recorrido paso a paso

Los números corresponden a los de la figura.

Pasos	Qué ocurre
1 a 3	El Coordinador envía formalizarContrato con el evento, el monto del anticipo recibido y el comprobante. El interceptor comprueba que el rol sea Coordinador, que es el actor documentado del CU-03, y pasa el comando al handler con userId y rol de la sesión.
4 a 7	El handler carga el Evento con IEventoRepository. La implementación lee de la base de datos. El handler solo conoce la interfaz.
8 a 11	Empieza la Transacción 1. Evento.formalizarContrato comprueba las cuatro condiciones de BR-07. Si todas se cumplen, cambia su estado y registra ContratoFormalizado y CambioRegistrado. Si falta alguna, lanza CondicionFormalizacionFaltanteException y no cambia nada.
12 a 17	El handler extrae los eventos del agregado, guarda el Evento y confirma. Aquí termina la Transacción 1.
18 y 19	El handler despacha los dos eventos en memoria y devuelve Result.exito al Coordinador, sin esperar las reacciones.
20 a 25	Transacción 2. ManejadorContratoFormalizado recibe el evento, crea la CuentaDePagos con subtotal, anticipo pactado y anticipo pagado, la guarda y confirma.
26 a 30	Transacción 3. ManejadorCambioRegistrado agrega un registro nuevo a la auditoría. No modifica los anteriores.

Límites transaccionales

Hay tres transacciones y ninguna comparte agregado con otra. La primera protege a Evento, la segunda a CuentaDePagos y la tercera a la tabla de auditoría. El orden entre la segunda y la tercera no está definido y no importa: son independientes.

Consistencia eventual

Entre el paso 17 y el paso 25 existe un contrato formalizado sin cuenta de pagos. Es una ventana corta pero real, y es el precio de no tener una transacción que abarque Evento y CuentaDePagos, algo que el mapa de dominio descarta de forma explícita. Durante esa ventana, una consulta de saldo devuelve CUENTA_NO_DISPONIBLE (sección 6.5). Si el proceso cae después del paso 17 y antes del despacho, la cuenta no se crea; ese riesgo se registra en ADR-004.

Por qué userId y rol viajan como metadatos de auditoría

La validación del rol ya ocurrió en el interceptor (ADR-002). Evento.formalizarContrato no usa userId ni rol para decidir nada: formaliza igual sin importar quién lo pidió. Entonces, ¿por qué recibirlos?

Porque FR-17 exige registrar quién hizo cada cambio, y el registro lo escribe ManejadorCambioRegistrado después del commit, en otra transacción y sin la sesión del usuario a la mano. La única forma de que el manejador sepa quién fue es que la identidad viaje dentro del evento. Por eso entran al agregado como datos simples, el agregado los copia en CambioRegistrado y en el payload de ContratoFormalizado (D10), y el manejador los escribe en la auditoría.

Dos detalles acompañan esta decisión. Primero, userId y rol los pone la Service Layer desde el contexto de seguridad y no un campo que mande el cliente, para que nadie pueda firmar un cambio a nombre de otro. Segundo, la alternativa de que el manejador consulte la sesión directamente se descartó, porque lo ataría a una sesión que puede no existir cuando el evento se procesa. La misma razón deja una pregunta abierta para los cambios que dispara el Scheduler, donde no hay usuario (capítulo 10.2).

8.3 Diagrama 2: CU-04 Procesar cambio de alcance (cambio extemporáneo no viable)

Campo	Contenido
Identificador	Secuencia_CU04_Alternativa
Caso de uso	CU-04 Procesar cambio de alcance. Actores: Cliente y Coordinador.
Reglas	BR-02 (límite de 7 días), BR-12 (cambios fuera de plazo) y BR-08 (costo, en la variante).
Operación	ProcesarCambioAlcanceCommand en ProcesarCambioAlcanceHandler (S-04, C-02).
Objetivo	Mostrar un desenlace alternativo previsto por el negocio: un cambio a menos de 7 días y sin viabilidad queda como “No garantizado”.

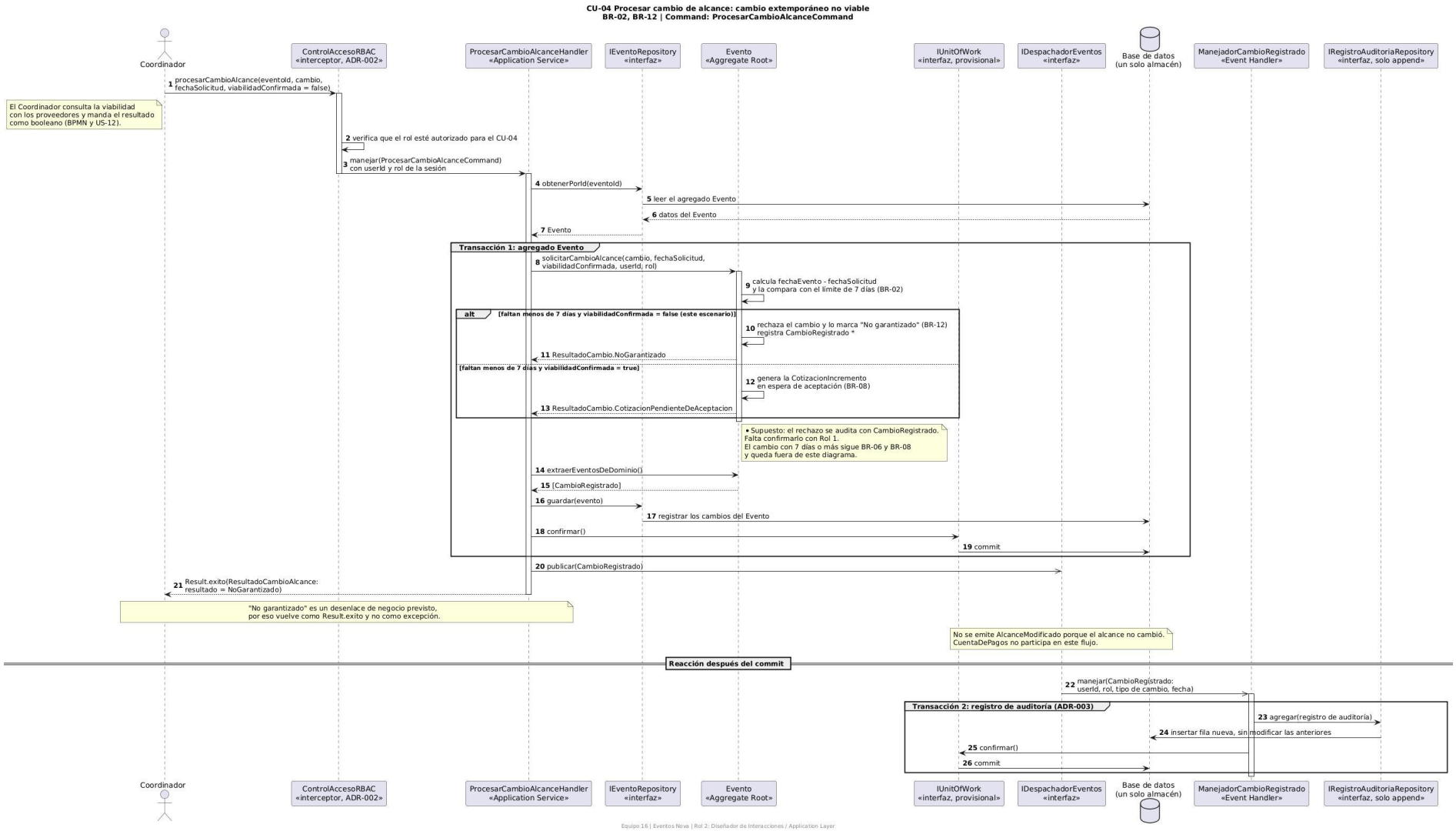


Figura 2: Secuencia CU-04, cambio extemporáneo no viable (Secuencia_CU04_Alternativa).

Recorrido paso a paso

Pasos	Qué ocurre
1 a 3	El Coordinador envía el cambio con la fecha de solicitud y viabilidadConfirmada = false, resultado de consultar a los proveedores (BPMN y US-12). El interceptor verifica el rol y el handler recibe el comando con userId y rol.
4 a 7	El handler carga el Evento por su interfaz de repositorio.
8 a 13	Empieza la Transacción 1. Evento.solicitarCambioAlcance calcula fechaEvento - fechaSolicitud y la compara con el límite de 7 días (BR-02). Con menos de 7 días y sin viabilidad, rechaza el cambio y lo marca “No garantizado” (BR-12). Con viabilidad, genera la CotizacionIncremento en espera de aceptación (BR-08). El cambio con 7 días o más sigue BR-06 y BR-08 y queda fuera de este diagrama.
14 a 19	El handler extrae los eventos, guarda el Evento con el cambio marcado y confirma. Aquí termina la Transacción 1.
20 y 21	El handler despacha CambioRegistrado y devuelve Result.exito con resultado = NoGarantizado.
22 a 26	Transacción 2. ManejadorCambioRegistrado agrega el registro a la auditoría.

Límites transaccionales y consistencia eventual

Aquí solo hay dos transacciones: la de Evento y la de la auditoría. No se emite AlcanceModificado, porque el alcance no cambió, y por eso CuentaDePagos no participa. La consistencia eventual se reduce al registro de auditoría, que se escribe después del commit. El supuesto de que el rechazo se audita con CambioRegistrado falta confirmarlo con Rol 1 (capítulo 10.1).

Por qué “No garantizado” se modela como Result de dominio

Rechazar un cambio fuera de plazo es lo que el negocio espera que pase en ese caso. Fase 1 lo lista como uno de los tres resultados posibles de CU-04, junto al cambio aplicado y la cotización en espera. No es una falla del sistema ni una invariante violada. Hay cuatro razones concretas para tratarlo como resultado y no como excepción.

1. **Hay estado que guardar.** El cambio queda marcado “No garantizado” y debe auditarse. Con una excepción, el flujo se interrumpe antes de guardar y confirmar, y ese registro se perdería o habría que reconstruirlo con un manejo de compensación.

2. **Quien llama necesita la respuesta.** El Coordinador tiene que decirle al Cliente que su cambio no está garantizado. Un resultado tipado se lo entrega. Una excepción obligaría a leer el tipo de error para obtener una respuesta normal.
3. **La decisión sigue siendo del dominio.** Evento devuelve ResultadoCambio.NoGarantizado. El servicio solo lo envuelve en Result.exito, sin evaluar fechas ni viabilidad.
4. **Es coherente con la estrategia D4.** Las excepciones se reservan para invariantes violadas, como en CU-03, donde faltar una condición bloquea la formalización y el agregado no cambia. Allí no hay nada que persistir, y la excepción es la forma natural de pararlo.

8.4 Coherencia con las tablas de Service Layer y CQRS

Los diagramas dibujan solo dos de las operaciones. CU-01, CU-02 y CU-05 siguen el mismo recorrido de la sección 4.4 y no se dibujaron porque la guía pide dos diagramas. Se comprobó lo siguiente:

- Todos los participantes de las dos figuras existen en las tablas de los capítulos 4 y 6 con el mismo nombre (sección 8.1).
- Los dos handlers dibujados son los mismos que figuran como S-03 y S-04 y como C-01 y C-02.
- Ninguna llamada va del servicio a una implementación de base de datos. Siempre pasa por una interfaz.
- Ninguna transacción abarca Evento y CuentaDePagos.

Los archivos fuente son Secuencia_CU03_Exito.puml y Secuencia_CU04_Alternativa.puml. Para regenerar las imágenes basta con PlantUML y Graphviz.

9. Registros de decisiones de arquitectura (ADR)

Este capítulo reúne los cuatro ADR del proyecto. Los tres primeros están aceptados. ADR-004 es una propuesta de la capa de aplicación.

ID	Título	Estado	Trazabilidad principal
ADR-001	Patrón de lógica de negocio: Domain Model	Aceptada	BR-04, BR-02, BR-07, BR-12, BR-14
ADR-002	Mecanismo de control de acceso (RBAC)	Aceptada	BR-16, BR-17, US-16, US-10
ADR-003	Estrategia de auditoría e historial inmutable	Aceptada	FR-17, NFR-01, BN-02, BN-12, BR-08
ADR-004	Despacho de eventos de dominio en memoria después de confirmar la transacción	Propuesta	CU-03, CU-04, CU-05, FR-17

9.1 ADR-001: Patrón de lógica de negocio, Domain Model

Campo	Contenido
Estado	Aceptada (12 de octubre de 2026).
Contexto	Eventos Nova requiere orquestar reglas complejas y dependientes del tiempo, como la validación del SLA de 48 horas (BR-04), el límite de 7 días para cambios (BR-02) y la verificación de las 4 condiciones para formalizar contratos (BR-07).
Alternativas	Transaction Script (descartada por duplicidad de condicionales), Table Module (descartada por rigidez y acoplamiento a tablas), Domain Model (elegida).
Decisión	Se adopta Domain Model. La entidad Evento encapsulará las reglas de negocio, validando estados e invariantes antes de la persistencia.
Consecuencias	Código cohesivo y testeable en aislamiento (SOLID). Mayor número de clases y necesidad de blindar la Service Layer para evitar modelos anémicos.
Trazabilidad	BR-04, BR-02, BR-07, BR-12, BR-14.

9.2 ADR-002: Mecanismo de control de acceso (RBAC)

Campo	Contenido
Estado	Aceptada (12 de octubre de 2026).
Contexto	El negocio exige exclusividad de funciones: solo Coordinación puede cancelar proveedores (BR-17) y solo Administración puede autorizar excepciones de pago (BR-16).
Alternativas	Validación en Dominio (descartada porque mezclaría seguridad e identidad con reglas comerciales), Validación en Service Layer (elegida).
Decisión	La validación de roles de usuario (Cliente, Coordinador, Administración, Proveedor) se inyectará como middlewares o interceptores en la Service Layer, manteniendo al Dominio agnóstico del usuario en sesión.
Consecuencias	Respeta SRP y mantiene el modelo desacoplado de sesiones de usuario. La Service Layer debe asegurar la intercepción en cada endpoint.
Trazabilidad	BR-16, BR-17, US-16, US-10.

9.3 ADR-003: Estrategia de auditoría e historial inmutable

Campo	Contenido
Estado	Aceptada (12 de octubre de 2026).
Contexto	Se necesita una fuente única de la verdad para registrar usuario, fecha, tipo de cambio y costo asociado (FR-17, NFR-01), evitando cotizaciones desfasadas.
Alternativas	Event Sourcing (descartada por sobreingeniería para el alcance del proyecto), Tablas Append-only relacionales (elegida).
Decisión	El historial de cambios operará sobre tablas relacionales dedicadas de solo anexado (“Append-only”). Las modificaciones generan un nuevo registro auditable sin sobrescribir el historial previo.
Consecuencias	Trazabilidad íntegra para auditorías cumpliendo FR-17. Costo moderado de almacenamiento en base de datos relacional.
Trazabilidad	FR-17, NFR-01, BN-02, BN-12, BR-08.

9.4 ADR-004: despacho de eventos en memoria (propuesta del Rol 2)

Campo	Contenido
Título	Despacho de eventos de dominio en memoria después de confirmar la transacción del agregado.
Estado	Propuesta. Pendiente de aceptación del equipo.
Relacionada con	ADR-001 y ADR-003.
Contexto	El mapa de dominio descarta la consistencia transaccional fuerte entre Evento y CuentaDePagos. Exige que Evento emita eventos y que CuentaDePagos reaccione de forma asíncrona. El registro de auditoría se alimenta también de eventos (CambioRegistrado). Falta decidir cómo viaja un evento desde que el agregado lo registra hasta que alguien reacciona. ADR-003 ya descartó Event Sourcing por ser una sobreingeniería para el nivel del proyecto.
Alternativas	A. Eventos en memoria, despachados dentro del proceso después del commit. Es lo más simple y se prueba fácil. Su costo: si el proceso cae entre el commit y el despacho, el evento se pierde. B. Mensajería asíncrona con un broker. Es lo más parecido a un sistema real, pero ningún documento menciona esa infraestructura y excede el alcance. C. Guardar el evento en una tabla durante la misma transacción y procesarlo desde ahí. Evita la pérdida de A con poca infraestructura, pero agrega una tabla y un procesador.
Decisión	Se elige la alternativa A. Cada reacción corre en su propia transacción, lo que cumple el “asíncrono” del mapa sin infraestructura que las fuentes no mencionan.
Consecuencias	Beneficios. Diseño simple, fácil de probar, sin dependencias nuevas. Costos y riesgos. Un evento puede perderse si el proceso cae antes del despacho. La pérdida pesa más en la auditoría (FR-17) que en las finanzas, y C sería la evolución natural. No hay política de reintentos, porque las fuentes no la definen. Existe una ventana en la que el contrato está formalizado y la cuenta aún no existe, y la consulta de saldo devuelve CUENTA_NO_DISPONIBLE.
Trazabilidad	CU-03, CU-04 y CU-05. BR-07, BR-02, BR-12 y BR-09. FR-17. ADR-001 y ADR-003. Decisión D1. Diagramas Secuencia_CU03_Exito y Secuencia_CU04_Alternativa.

Otras decisiones de este diseño podrían registrarse como ADR si el equipo lo prefiere: la estrategia de errores y resultados (D4), la transacción por agregado (D5) y la

selección de operaciones CQRS (D2). Quedan documentadas en los capítulos 4 y 6 y no se convirtieron en ADR para evitar duplicar contenido.

10. Hallazgos y puntos pendientes

La guía pide registrar como hallazgo cualquier contradicción o dato faltante, y no inventar reglas para acomodar la arquitectura. Esta sección reúne lo que apareció al diseñar la capa de aplicación.

10.1 Propuestas que cambian el trabajo de Rol 1

1. Evento.marcarComoPagado() y Evento.evaluarLiquidacion(ahora) no están en la ficha del agregado. El primero es la decisión D7 y el segundo es propuesta del Rol 2.
2. formalizarContrato() aparece sin parámetros en el mapa. Aquí recibe el monto del anticipo, el comprobante, userId y rol. Lo mismo pasa con solicitarCambioAlcance().
3. Las cuatro condiciones de BR-07 (D12) sustituyen la invariante del mapa. Según esa lista, los 15 días de BR-01 se validan en CU-01 y no al formalizar. La ficha del agregado debe actualizarse.
4. En el mapa, la cancelación a 7 días aparece con BR-02. Según el BRD, BR-02 es el límite de cambios y BR-14 cubre la notificación a 7 días y la retención de gastos comprometidos. Hay que corregir la etiqueta y registrarlo como ajuste de trazabilidad.
5. El payload de ContratoFormalizado lleva clienteId, pero el mapa no guarda cliente en CuentaDePagos. Hay que decidir si la cuenta lo conserva o si el campo sobra.
6. La fórmula documentada resta anticipo pactado más pagos realizados. Al crear la cuenta llega anticipoPagado. Hay que decidir cómo se registra, como pago realizado o como anticipo, porque cambia lo que muestra el saldo. Además, MontoContractual en el diagrama solo tiene subtotal y pagosRealizados, y la fórmula necesita también incrementos aprobados y anticipo pactado.
7. Fase 1 dice que Evento recalcula el MontoContractual, y el mapa lo pone dentro de CuentaDePagos. Este informe sigue el mapa.
8. El rechazo “No garantizado” se audita con CambioRegistrado. Es un supuesto del Rol 2 que falta confirmar.

10.2 Datos que las fuentes no traen

- Dónde se guarda el cambio rechazado como “No garantizado”: el mapa no define esa entidad.
- Qué operación aprueba una CotizacionIncremento. No está entre CU-01 y CU-05, pero H-04 depende de ella.
- Qué efecto tiene AlcanceModificado en CuentaDePagos.
- Qué hace CuentaDePagos con EventoCancelado (cálculo de devolución). Queda fuera de los casos elegidos.
- El detalle de BR-10 y qué pasa si el proveedor responde después del SLA.
- Quién consume AsignacionProveedorVencida. No hay notificación definida.
- Qué agregado registra la excepción de pago que autoriza Administración.

- Cómo se enteran las condiciones 1 a 3 de BR-07 dentro de Evento: no hay operaciones previas documentadas.
- Qué roles pueden ejecutar cada Query.
- Si en CU-01 un Evento con anticipación insuficiente se persiste o no. Aquí se asume que no.
- Qué userId y rol figuran en los cambios que dispara el Scheduler, donde no hay una persona.
- Qué pasa si confirmar() falla o si un manejador de eventos falla: no hay política de reintento.

10.3 Riesgos de diseño aceptados

Riesgo	Dónde aparece	Estado
Pérdida de un evento si el proceso cae entre el commit y el despacho	D1, ADR-004	Aceptado. C (tabla de eventos) queda como evolución.
Fórmula del saldo repetida en lectura y escritura	D2, sección 6.6	Aceptado. Hay que mantener ambas copias iguales.
Cuenta de pagos inexistente durante un instante tras formalizar	Secciones 6.5 y 8.2	Previsto. La consulta devuelve CUENTA_NO_DISPONIBLE.
Sin política de reintento entre manejadores	Secciones 4.11 y 10.2	Pendiente de definir por el equipo.

10.4 Actualización tras integrar los capítulos de Rol 1 y Rol 3

Los capítulos 1, 2, 3, 5 y 7 y el Anexo A llegaron después de redactar las secciones 10.1 a 10.4. Esas secciones se conservan tal como se escribieron. Aquí se anota qué cambió con la integración y qué inconsistencias nuevas aparecieron.

Hallazgos anteriores: estado actual

Hallazgo previo	Estado tras la integración
10.1, punto 1: marcarComoPagado() y evaluarLiquidacion(ahora) no estaban en la ficha.	Parcial. marcarComoPagado() ya aparece en el diagrama de clases (Anexo A). evaluarLiquidacion(ahora) no aparece en ningún lado.
10.1, punto 2: parámetros de formalizarContrato() y solicitarCambioAlcance().	Resuelto. El Anexo A los incluye con monto, comprobante, userId y rol.

Hallazgo previo	Estado tras la integración
10.1, punto 3: las cuatro condiciones de BR-07.	Resuelto. El capítulo 1 y la ficha de Evento (capítulo 3) las listan.
10.1, punto 4: etiquetas BR-02 y BR-14.	Resuelto en el capítulo 3, que usa BR-02 para el límite de cambios. La cancelación antes de 7 días ya no aparece como invariante de Evento, así que queda sin dueño documentado.
10.1, punto 5: clienteId en el payload de ContratoFormalizado.	Sigue pendiente. CuentaDePagos en el Anexo A no guarda clienteId.
10.1, punto 6: atributos de MontoContractual y registro del anticipo.	Parcial. El Anexo A ya trae subtotal, anticipo pactado, pagos realizados e incrementos. Sigue sin haber atributo para anticipoPagado.
10.1, punto 7: quién recalcula el MontoContractual.	Atenuado. El capítulo 2 ya no menciona ServiciosContratados y habla de recálculo financiero desacoplado. Conviene confirmar la redacción.
10.1, punto 8: el rechazo “No garantizado” se audita con CambioRegistrado.	Sigue pendiente.
10.2: quién aprueba una CotizacionIncremento.	Parcial. El capítulo 1 dice que el Cliente autoriza incrementos de costo. La operación que lo hace sigue sin estar entre CU-01 y CU-05.
10.3: quién lleva las etapas 5 y 7.	Resuelto. Las entregó Rol 3. Las interfaces provisionales de la sección 4.8 coinciden con las del capítulo 5, salvo las de lectura y IDespachadorEventos (sección 5.3).

10.5 Mitigaciones y Decisiones de Coordinación entre Capas

1. **Alineación de componentes:** Los participantes modelados en los diagramas de secuencia (ControlAccesoRBAC, IDespachadorEventos, handlers de CQRS y manejadores de eventos) corresponden a componentes de coordinación de la Capa de Aplicación, mientras que el Diagrama de Clases del Anexo A modela la estructura estática del dominio y los contratos de persistencia.
2. **Desacoplamiento de identidad:** La entidad CuentaDePagos no almacena directamente clienteId para mantener el agregado financiero libre de dependencias de sesión; el vínculo se resuelve de forma no acoplada a través de la referencia eventoId.

3. **Consistencia eventual de eventos:** La emisión asíncrona de eventos de dominio hacia la cuenta de pagos y hacia la tabla de auditoría append-only garantiza que cada Aggregate Root preserve su propia frontera transaccional atómica e independiente.
4. **Generalización de interfaces:** La interfaz genérica IRepository<T> actúa como abstracción base dentro de la infraestructura en C#, mientras que el catálogo especializado expone los métodos concretos del negocio (IEventoRepository, ICuentaDePagosRepository, IRegistroAuditoriaRepository).

11. Conclusiones de la capa de aplicación

La capa de aplicación de Eventos Nova quedó reducida a una función concreta: recibir una intención, cargar un agregado, pedirle al dominio que decida, guardar y avisar. Los cinco servicios de los casos de uso, los dos del Scheduler y los cuatro manejadores de eventos siguen ese recorrido, y en ningún diagrama aparece un condicional de negocio dentro de un handler. Las reglas BR-07, BR-02 y BR-12, que son las que se dibujaron, se decidieron dentro de Evento.

Esa separación descansa en una decisión que tiene costo: la consistencia eventual entre Evento y CuentaDePagos. El mapa de dominio la exige y el diseño la respeta con una transacción por agregado y con eventos en memoria que se despachan después del commit. Eso trae tres consecuencias que el equipo debe poder explicar. Un evento puede perderse si el proceso cae entre el commit y el despacho. La cuenta de pagos no existe durante un instante después de formalizar, por lo que la consulta de saldo puede responder CUENTA_NO_DISPONIBLE. Y no hay política de reintentos, porque ningún documento la define y no se inventó una.

CQRS se aplicó en pequeño. Los dos Commands están donde viven las reglas más fuertes del caso (BR-07 y las reglas de tiempo de CU-04) y las dos Queries leen un saldo y un SLA sin tocar los agregados. Todo usa un solo almacén. La fórmula del saldo existe dos veces, una en el dominio y otra en la lectura, y ese es el precio de leer sin cargar el agregado. Mientras la fórmula documentada no cambie no pasa nada, y si cambia hay que tocar dos lugares.

Varias piezas dependen de que Rol 1 acepte cambios en sus fichas: marcarComoPagado(), evaluarLiquidacion(ahora), los parámetros nuevos de formalizarContrato() y solicitarCambioAlcance(), la lista de condiciones de BR-07 y las etiquetas de BR-02 y BR-14. Mientras no se validen aparecen marcadas como propuesta. Tampoco pueden cerrarse sin los datos que la documentación no trae, listados en la sección 10.2.

Con lo que hay hoy, el equipo puede explicar dónde está cada regla, quién abre y cierra cada transacción, por qué un cambio rechazado es un resultado y no una excepción, y por qué la identidad del usuario viaja dentro de los eventos. Lo que falta por sostener depende de las etapas 5 y 7 y de las respuestas pendientes de la sección 10.2.

Anexo A: Diagrama de clases UML de la arquitectura (Entregable 02)

El siguiente modelo estructural integra la Capa de Dominio, la Capa de Aplicación y las abstracciones de Infraestructura respetando los límites de contexto y el Lenguaje Ubicuo.

Código fuente: Diagrama_Clases_EventosNova.puml. La imagen se generó con PlantUML a partir de ese archivo, sin cambios.

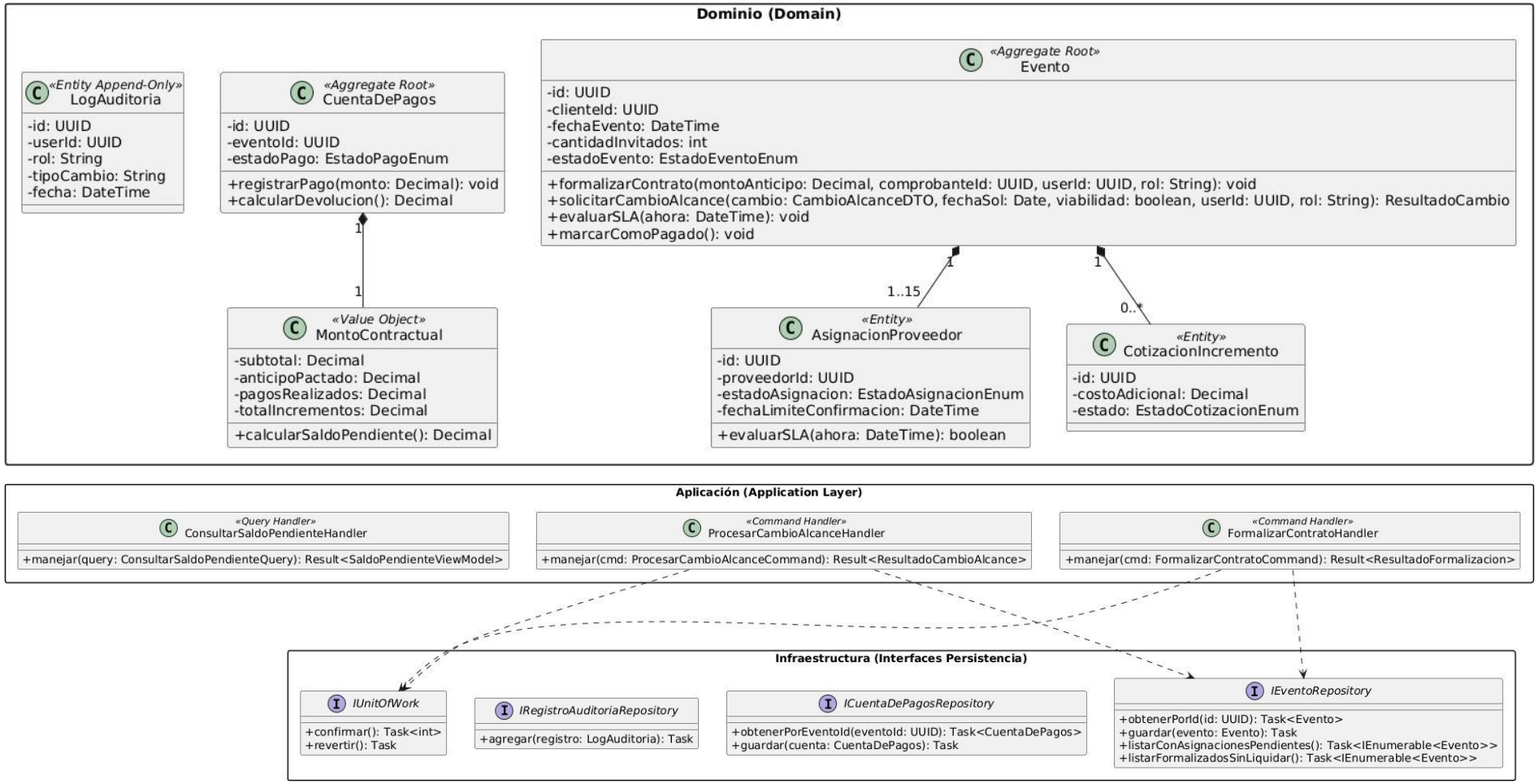


Figura 3: Diagrama de clases UML de la arquitectura (Dominio, Aplicación e Infraestructura).

A.1 Código fuente PlantUML

```
@startuml Diagrama_Clases_EventosNova
skinparam classAttributeIconSize 0
skinparam packageStyle rectangle
skinparam roundcorner 5

package "Dominio (Domain)" {
    class Evento <<Aggregate Root>> {
        - id: UUID
        - clienteId: UUID
        - fechaEvento: DateTime
        - cantidadInvitados: int
        - estadoEvento: EstadoEventoEnum
        + formalizarContrato(montoAnticipo: Decimal, comprobanteId: UUID, userId: UUID, rol: String): void
        + solicitarCambioAlcance(cambio: CambioAlcanceDTO, fechaSol: Date, viabilidad: boolean,
            userId: UUID, rol: String): ResultadoCambio
        + evaluarSLA(ahora: DateTime): void
        + marcarComoPagado(): void
    }

    class AsignacionProveedor <<Entity>> {
        - id: UUID
        - proveedorId: UUID
        - estadoAsignacion: EstadoAsignacionEnum
        - fechaLimiteConfirmacion: DateTime
        + evaluarSLA(ahora: DateTime): boolean
    }

    class CotizacionIncremento <<Entity>> {
        - id: UUID
        - costoAdicional: Decimal
        - estado: EstadoCotizacionEnum
    }

    class CuentaDePagos <<Aggregate Root>> {
        - id: UUID
        - eventoId: UUID
        - estadoPago: EstadoPagoEnum
        + registrarPago(monto: Decimal): void
```

```
        + calcularDevolucion(): Decimal
    }

    class MontoContractual <<Value Object>> {
        - subtotal: Decimal
        - anticipoPactado: Decimal
        - pagosRealizados: Decimal
        - totalIncrementos: Decimal
        + calcularSaldoPendiente(): Decimal
    }

    class LogAuditoria <<Entity Append-Only>> {
        - id: UUID
        - userId: UUID
        - rol: String
        - tipoCambio: String
        - fecha: DateTime
    }

    Evento "1" *-- "1..15" AsignacionProveedor
    Evento "1" *-- "0..*" CotizacionIncremento
    CuentaDePagos "1" *-- "1" MontoContractual
}

package "Aplicación (Application Layer)" {
    class FormalizarContratoHandler <<Command Handler>> {
        + manejar(cmd: FormalizarContratoCommand): Result<ResultadoFormalizacion>
    }

    class ProcesarCambioAlcanceHandler <<Command Handler>> {
        + manejar(cmd: ProcesarCambioAlcanceCommand): Result<ResultadoCambioAlcance>
    }

    class ConsultarSaldoPendienteHandler <<Query Handler>> {
        + manejar(query: ConsultarSaldoPendienteQuery): Result<SaldoPendienteViewModel>
    }
}

package "Infraestructura (Interfaces Persistencia)" {
    interface IEventoRepository {
        + obtenerPorId(id: UUID): Task<Evento>
```

```
        + guardar(evento: Evento): Task
        + listarConAsignacionesPendientes(): Task<IEnumerable<Evento>>
        + listarFormalizadosSinLiquidar(): Task<IEnumerable<Evento>>
    }

    interface ICuentaDePagosRepository {
        + obtenerPorEventoId(eventoId: UUID): Task<CuentaDePagos>
        + guardar(cuenta: CuentaDePagos): Task
    }

    interface IRegistroAuditoriaRepository {
        + agregar(registro: LogAuditoria): Task
    }

    interface IUnitOfWork {
        + confirmar(): Task<int>
        + revertir(): Task
    }
}

FormalizarContratoHandler ..> IEventoRepository
FormalizarContratoHandler ..> IUnitOfWork
ProcesarCambioAlcanceHandler ..> IEventoRepository
ProcesarCambioAlcanceHandler ..> IUnitOfWork
@enduml
```

Anexo B: Código Base Boilerplate en C# (Entregable 06)

Estructura de carpetas del repositorio:

```
src/
├── EventosNova.Domain/
│   ├── Aggregates/
│   │   ├── Evento.cs
│   │   └── CuentaDePagos.cs
│   ├── Entities/
│   │   ├── AsignacionProveedor.cs
│   │   ├── CotizacionIncremento.cs
│   │   └── LogAuditoria.cs
│   └── ValueObjects/
│       └── MontoContractual.cs
└── EventosNova.Infrastructure/
    ├── Interfaces/
    │   ├── IRepository.cs
    │   ├── IEventoRepository.cs
    │   ├── ICuentaDePagosRepository.cs
    │   ├── IRegistroAuditoriaRepository.cs
    │   └── IUnitOfWork.cs
    └── Persistence/
        ├── EventosNovaDbContext.cs
        └── UnitOfWork.cs
```

Contratos e interfaces de persistencia (C#):

```
namespace EventosNova.Infrastructure.Interfaces;

public interface IRepository<T> where T : class
{
    Task<T?> GetByIdAsync(Guid id);
    Task<IEnumerable<T>> GetAllAsync();
    Task AddAsync(T entity);
    void Update(T entity);
}
```

```
public interface IEventoRepository : IRepository<Domain.Aggregates.Evento>
{
    Task<Domain.Aggregates.Evento?> ObtenerPorIdAsync(Guid eventoId);
    Task GuardarAsync(Domain.Aggregates.Evento evento);
    Task<IEnumerable<Domain.Aggregates.Evento>> ListarConAsignacionesPendientesAsync();
    Task<IEnumerable<Domain.Aggregates.Evento>> ListarFormalizadosSinLiquidarAsync();
}

public interface ICuentaDePagosRepository : IRepository<Domain.Aggregates.CuentaDePagos>
{
    Task<Domain.Aggregates.CuentaDePagos?> ObtenerPorEventoIdAsync(Guid eventoId);
    Task GuardarAsync(Domain.Aggregates.CuentaDePagos cuenta);
}

public interface IRegistroAuditoriaRepository
{
    Task AgregarAsync(Domain.Entities.LogAuditoria registro);
}

public interface IUnitOfWork : IDisposable
{
    Task<int> ConfirmarAsync(CancellationToken cancellationToken = default);
    Task RevertirAsync();
}
```

Anexo C: Bitácora Oficial de IA (Entregable 05)

ID	Prompt Enviado a la IA	Respuesta Relevante de la IA	Decisión Humana / Ajuste Aplicado
PRM-01	“Genera una interfaz genérica de repositorio en C# que incluya métodos asíncronos y guardado de cambios directo en cada método.”	Propuse IRepository incluyendo un método SaveAsync() dentro de la propia interfaz genérica.	**Rechazado:** Se eliminó SaveAsync() de repositorios individuales y se trasladó el control de transacciones exclusivamente a IUnitOfWork.
PRM-02	“¿Cómo puedo modelar en un repositorio la regla de negocio que limita a un evento a no más de 15 proveedores?”	Sugirió incluir la validación dentro de un método AddProvider() directamente en la clase concreta SqlRepository.	**Ajustado:** La validación del límite de 15 proveedores se delegó como invariante interna del agregado Evento, manteniendo el repositorio libre de reglas.
PRM-03	“Escribe las interfaces en C# para manejar la expiración de cotizaciones de proveedores después de 48 horas.”	Proporcionó el método GetExpiredProposalsAsync() dentro de un repositorio de proveedores.	**Aceptado:** Se adaptó como método de consulta en IEventoRepository para ser invocado periódicamente por el servicio en segundo plano (Scheduler).
PRM-04	“Genera un diagrama UML en PlantUML para mostrar la relación entre IRepository y Entity Framework DbContext.”	Generó un diagrama acoplando las interfaces de infraestructura directamente a clases concretas de Entity Framework (DbContext).	**Ajustado:** Se reestructuró el modelo para mostrar exclusivamente abstracciones (Interfaces) y Entidades de Dominio, preservando Inversión de Dependencias (DIP).